API SECURITY RISK ANALYSIS

Future Interns — Cyber Security Internship | Task 3

Target: JSONPlaceholder REST API **Tool:** Postman **Date:** 7 October 2026

Prepared by: Network Terminator

1. Executive Summary

This report documents a basic API security assessment of the JSONPlaceholder REST API using Postman. The assessment covered authentication, authorization, data exposure, rate limiting, and input validation.

Testing was limited to read-only GET requests and benign input values. No brute-force, denial-of-service, destructive, or high-volume testing was performed.

Important context: JSONPlaceholder is a public fake API intended for testing and prototyping, and its returned data is synthetic. The observations in this report are therefore treated as security observations rather than confirmed production vulnerabilities.

2. Scope

Item	Details
Target	https://jsonplaceholder.typicode.com
Endpoints	GET /users; GET /users/1; GET /users/2; GET /users?id=abc
Assessment areas	Authentication; Authorization; Data Exposure; Rate Limiting; Input Validation
Testing approach	Read-only, non-intrusive requests and benign input

3. Tools Used

Postman — API requests and response inspection. Web Browser — target/API review. GitHub — evidence and documentation.

4. Methodology

1. Establish a baseline response. 2. Check unauthenticated access. 3. Compare resource IDs. 4. Review returned fields. 5. Inspect response headers for rate-limit indicators. 6. Test an unexpected non-numeric ID. 7. Capture evidence.

5. Assessment Summary

Area	Result	Assessment
Authentication	/users/1 returned 200 OK without authentication.	Informational / Contextual
Authorization	/users/1 and /users/2 directly accessible.	Informational / Contextual
Data Exposure	Multiple user-related fields returned.	Low / Contextual
Rate Limiting	No explicit rate-limit headers observed.	Informational
Input Validation	id=abc returned 200 OK with [].	Informational

6. Baseline API Test

Request
GET https://jsonplaceholder.typicode.com/users

Result
HTTP 200 OK. The response contained JSON user records. Postman showed approximately: Status 200 OK, Time 89 ms, Size 2.67 KB.

Observation
The endpoint responded successfully and returned structured JSON data.

Evidence
Baseline/users-baseline.png

7. Authentication Assessment

Request
GET https://jsonplaceholder.typicode.com/users/1

Result
HTTP 200 OK without an Authorization header.

Observation
No authentication mechanism was required for this endpoint.

Assessment
Informational / Contextual.

Security consideration
Unauthenticated access is expected for this public demonstration API. In a production API containing private information, the same behavior could allow unauthorized access to protected resources.

Recommendation
Require appropriate authentication for sensitive resources and validate authentication tokens server-side.

Evidence
Authentication/authentication-no-auth.png

8. Authorization Assessment

Requests
GET https://jsonplaceholder.typicode.com/users/1
GET https://jsonplaceholder.typicode.com/users/2

Result
Both resources were accessible without authentication.

Observation
Changing the resource ID allowed another public record to be retrieved.

Assessment
Informational / Contextual.

Important limitation
This is not classified as a confirmed BOLA/IDOR vulnerability because JSONPlaceholder is intentionally public and uses synthetic data.

Recommendation
Production APIs should perform server-side object-level authorization checks for sensitive resources.

Evidence
Authorization/user-2-access.png

9. Data Exposure Assessment

Request
GET https://jsonplaceholder.typicode.com/users/1

Observed fields
id, name, username, email, address, street, suite, city, zipcode, geo, phone, website, company.

Observation
Multiple categories of user-related information are returned in one response.

Assessment
Low / Contextual.

Security consideration
The returned data is synthetic. In a production API, unnecessary fields could increase the impact of unauthorized access.

Recommendation
Return only the fields required by the client and protect sensitive fields with authorization.

Evidence
Data Exposure/user-data-exposure.png

10. Rate Limiting Assessment

Request
GET https://jsonplaceholder.typicode.com/users/1

Result
No explicit X-RateLimit-Limit, X-RateLimit-Remaining, RateLimit-Limit, RateLimit-Remaining, or Retry-After headers were observed.

Observation
No explicit rate-limiting information was visible in the sampled response.

Assessment
Informational.

Limitation
The absence of these headers does not prove that rate limiting is absent. No high-volume testing was performed.

Recommendation
Production APIs should implement suitable rate limiting and abuse monitoring.

Evidence
RateLimiting/rate-limit-headers.png

11. Input Validation Assessment

Request
GET https://jsonplaceholder.typicode.com/users?id=abc

Result
HTTP 200 OK with an empty JSON array: []

Observation
The unexpected value was handled without a server-side error. No abnormal or injection behavior was observed during this benign test.

Assessment
Informational.

Recommendation
Validate parameter types and formats server-side and use consistent status codes for invalid input.

Evidence
InputValidation/invalid-id-input.png

12. Recommendations

1. Implement strong authentication for sensitive resources.
2. Apply server-side object-level authorization.
3. Return only the minimum data required by the client.
4. Implement appropriate rate limiting.
5. Validate API parameters server-side.
6. Use secure and consistent HTTP status codes.
7. Monitor API activity for abuse.
8. Avoid exposing internal implementation details.
9. Regularly test APIs against OWASP API Security risks.
10. Review authorization controls for sensitive endpoints.

13. Limitations

• JSONPlaceholder is a public fake REST API designed for testing and prototyping.
• Returned data is synthetic.
• No authenticated/private functionality was available for testing.
• No real authorization boundary could be evaluated.
• No high-volume requests were performed.
• Rate limiting was assessed only through passive header inspection.
• Testing was limited to selected endpoints and benign requests.

14. Conclusion

The assessment demonstrated a structured approach to API security testing using Postman. Several security-relevant behaviors were documented, but none were treated as confirmed exploitable vulnerabilities because the target is intentionally public and uses synthetic data.

15. Evidence Index

Test	Evidence file
Baseline	Baseline/users-baseline.png
Authentication	Authentication/authentication-no-auth.png
Authorization	Authorization/user-2-access.png
Data Exposure	Data Exposure/user-data-exposure.png
Rate Limiting	RateLimiting/rate-limit-headers.png
Input Validation	InputValidation/invalid-id-input.png